

Debugging

Topics Covered:

- What is debugging?
 - Step-by-step execution
 - Environment preparation
 - The debugger statement
 - The Resume option
 - Code debugging without the debugger statement
 - The Step Over option
 - The Step Into option
 - The Call Stack option
 - Viewing and modifying variables
 - The Step Out option
 - Measuring code execution time
-

Testing and Debugging Your Code

Errors in programming are natural and expected. Initially, syntax errors dominate due to a lack of familiarity with programming languages. These are easier to fix as interpreters often provide feedback. However, logical errors, which cause programs to behave unexpectedly, require detailed analysis and testing.

Example 1: Logical Error in Average Calculation

javascript

Copy code

```
function average(a, b) {  
    return a + b / 2;  
}
```

```
console.log(average(2, 10)); // -> 7 (expected: 6)
```

```
console.log(average(5, 5)); // -> 7.5 (expected: 5)
```

Issue: Misinterpreted order of operations.

Explanation: The division ($b / 2$) executes first, followed by addition.

Correction:

javascript

Copy code

```
return (a + b) / 2;
```

This fixes the logic to align with the programmer's intent.

Example 2: Logical Error in Finding the Largest Number

javascript

Copy code

```
function largest(a, b, c) {  
  if (a > b && a > c) {  
    return a;  
  } else if (b > a && b > c) {  
    return b;  
  } else {  
    return c;  
  }  
}
```

```
console.log(largest(1, 1, 2)); // -> 2
```

```
console.log(largest(1, 2, 3)); // -> 3
```

```
console.log(largest(3, 2, 1)); // -> 3
```

```
console.log(largest(2, 2, 1)); // -> 1 (Incorrect)
```

Issue: When two numbers are equal, the logic fails.

Correction: Replace $>$ with $\geq$ in conditional checks.

Debugging

Modern browsers come equipped with debugging tools that provide powerful functionality. Here's how to get started:

The debugger Statement

Adding debugger halts the program execution, allowing you to inspect the code.

Example:

javascript

Copy code

```
console.log("Before debugger");
```

```
debugger;
```

```
console.log("After debugger");
```

Step-by-Step Execution

Use the debugger to pause and control program execution step by step. Most browsers provide these options:

- **Resume / Continue:** Resumes normal execution.
 - **Step Into:** Moves into function calls for detailed inspection.
 - **Step Over:** Skips over function internals but continues execution.
 - **Step Out:** Exits the current function.
-

Setting Up the Debugging Environment

Create these files:

1. **index.html**

html

Copy code

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <script src="main.js"></script>
```

```
</head>
```

```
<body>
  <p>Test Site</p>
</body>
</html>
```

2. **main.js**

javascript

Copy code

```
function outer() {
  let name = "outer";
  let str = inner();
  return str;
}
```

```
function inner() {
  let name = "inner";
  return "Hello !";
}
```

```
console.log("before outer() call");
console.log(outer());
console.log("after outer() call");
```